

UNIDAD 11

PROGRAMACIÓN FUNCIONAL E INTRODUCCIÓN AL DESARROLLO WEB BACKEND

**Programación
CFGS DAW**

Autor: José Miguel Blázquez
2023/2024

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:



Importante



Atención



Interesante

ÍNDICE

1. Introducción.....	4
2. Programación funcional.....	4
2.1 Declarativo vs imperativo: ejemplos.....	5
2.2 Características de la programación funcional.....	6
2.3 Expresiones lambda (funciones anónimas).....	7
2.4 Streams.....	9
3. Otros lenguajes y paradigmas.....	13
3.1 Características de otros lenguajes multiparadigma con enfoque web.....	13
3.2 El tipado de datos.....	14
4. Introducción al desarrollo web backend con Java.....	16
4.1 Tecnologías backend en Java.....	17
4.2 Servlets.....	18
4.3 JSP.....	22

UD11. PROGRAMACIÓN FUNCIONAL E INTRODUCCIÓN AL DESARROLLO WEB BACKEND

1. INTRODUCCIÓN

En esta unidad vamos a conocer con un poco de detalle un estilo de programación del paradigma declarativo que, actualmente, está ganando popularidad en ámbitos donde la Programación Orientada a Objetos era la reina indiscutible. Sin embargo, y por el momento, no está llamada a sustituirla sino más bien a complementarla; nos referimos a la programación funcional, la cual está siendo añadida a lenguajes tradicionalmente imperativos u orientados a objetos como Java, JavaScript, C#...

En la siguiente parte de la unidad describiremos los lenguajes multiparadigma y hablaremos sobre una característica importante que diferencia un lenguaje de otro: el tipado de datos. Finalmente, en la parte final de la unidad hablaremos sobre el desarrollo “backend” y concretaremos ejemplos de desarrollo con Java y sus variantes para web (Java EE).

2. PROGRAMACIÓN FUNCIONAL

La Programación Funcional es un “paradigma de programación”, concretamente un tipo específico de la programación declarativa. Como ya estudiamos a principio de curso, un paradigma es una forma de pensar y de organizar un programa basado en una serie de principios (que detallaremos más adelante). Como comparación, el paradigma predominante estudiado hasta ahora era la Programación Orientada a Objetos, junto a la programación imperativa y la estructurada.

La programación funcional se basa fuertemente en el concepto de función matemática y otras propiedades y modelos, como el *cálculo lambda*. Ésta tiende a resultar en un código más conciso y expresivo; se presta para expresar y resolver problemas actuales de paralelización, computación distribuida, asincronía, eventos, ... en el ámbito de aplicaciones web podemos citar a Whatsapp y Facebook, por ejemplo. En el primero se utiliza Erlang, un lenguaje de programación funcional; y en el segundo se usa Haskell para el sistema antispam. Incluso en bases de datos ya que SQL es declarativo.

Dentro de los lenguajes funcionales, debemos distinguir entre:

- **Lenguajes funcionales puros:** fueron creados para seguir este paradigma, y no otro. En este grupo podemos encontrar lenguajes como Haskell o Miranda. El primero es utilizado por Facebook para manejar grandes datos y análisis.
 - **Lenguajes funcionales híbridos:** lenguajes que aceptan varios paradigmas, aunque les va bien con enfoques funcionales. En este grupo podemos hablar de Scal, Clojure o Lisp .
 - **Otros lenguajes:** inicialmente eran imperativos pero han modificado su sintaxis y estructura para aceptar algunas características funcionales. Es el caso de algunos de los lenguajes orientados a objetos más importantes de estos años, como: Java, C#, Javascript, PHP, Python, Ruby, Kotlin... Muchos de estos lenguajes no están 100% enfocados en la programación funcional, lo cual no es malo, de hecho, estos nos
-

permite hacer una transición de un paradigma a otro “en cualquier momento”, comúnmente de la programación orientada a objetos a la programación funcional.

Cabe resaltar que la programación funcional conlleva e incluye muchísimos más conceptos de los que vamos a tratar aquí, pero en esta unidad nos centraremos en lo más importante y utilizable en la práctica, aunque de forma resumida.

2.1 Declarativo vs imperativo: ejemplos

A diferencia de la programación estructurada y la POO (programación orientada a objetos), que son paradigmas “imperativos”, la programación funcional es “declarativa”. Esto quiere decir que en el estilo funcional expresamos nuestra lógica sin describir el “control de flujo” (bucles, condicionales, ...), enfocándonos en “qué” estamos haciendo en vez de “cómo” se está haciendo, que sería el enfoque imperativo. Como ejemplo comparativo, el lenguaje SQL también es declarativo.

Veamos un ejemplo de implementación: imperativo vs declarativo funcional

Se nos plantea el siguiente problema: conocer la cantidad de elementos mayores a 10 en la siguiente lista:

```
List<Integer> numeros = List.of(18, 6, 4, 15, 55, 78, 12, 9, 8);
```

Podemos resolver este problema utilizando bucles y condicionales: enfoque **imperativo**.

```
int contador = 0;
for(int numero : numeros){
    if(numero > 10){
        contador ++;
    }
}
System.out.println(contador);
```

Ahora vamos a verlo mediante el enfoque **de la programación funcional**:

```
long contador = numeros.stream().filter(num -> num > 10).count();
System.out.println(contador);
```

Aunque puede escribirse en una misma línea, es habitual separarlo para mejorar la legibilidad del código, veamos cómo quedaría el bloque anterior:

```
long contador = numeros.stream()
    .filter(num -> num > 10)
    .count();
System.out.println(contador);
```

Cómo podemos ver, con el enfoque funcional las líneas de código se reducen y el código se torna más legible. Se puede observar cómo primero se filtra por una condición y posteriormente se suma lo filtrado. Esta implementación podemos concluir que es funcional ya

que delega el control de flujos y condiciones a funciones (*filter* y *count*).

2.2 Características de la programación funcional

Al trabajar con programación funcional, nos toparemos con conceptos tales como:

- Funciones puras.
- Composición de funciones.
- Estado compartido.
- Inmutabilidad.
- Efectos secundarios.

Veamos cada uno de estos conceptos con mayor detalle:

FUNCIONES PURAS

Las funciones puras son aquellas que, dado el mismo conjunto de entradas, siempre devuelven el mismo conjunto de salidas y no producen efectos secundarios. Esto significa que son inmutables y deterministas. En la programación funcional, en lugar de usar bucles *for* y *while*, se utiliza la recursividad para la iteración.

COMPOSICIÓN DE FUNCIONES

La composición de funciones es el proceso de combinar dos o más funciones para ejecutarlas en secuencia y obtener un resultado específico. Por ejemplo:

```
> f(x)
> g(x)
> f(g(x))
```

En este caso combinamos la función *f* (de *x*) con la función *g* (de *x*).

ESTADO COMPARTIDO

El estado compartido se refiere a cualquier variable, objeto o espacio de memoria que exista en un ámbito accesible por múltiples partes del programa. En la programación orientada a objetos, los objetos se comparten entre ámbitos al agregar propiedades a otros objetos:

```
Materia materia = new Materia();
Usuario usuario = new Usuario();
usuario.materia = materia;
```

INMUTABILIDAD

Un objeto inmutable es aquel que no puede ser modificado una vez haya sido creado. Por ejemplo, una variable en Java no es un objeto inmutable, pero si es una variable de tipo *String* sí (recuerda la unidad 4).

```
String caracteres = "ABC";  
String nuevaCadana = caracteres.toLowerCase();
```

Podemos ver a los objetos inmutables como constantes, una vez nosotros asignamos un valor este no podrá ser modificado. La inmutabilidad también la podemos ver en este ejemplo: si queremos ordenar una lista de números enteros, lo que haremos será pasarle una función que nos devolverá una lista **nueva** pero ordenada sin tocar la lista original.

EFFECTOS SECUNDARIOS

Un efecto secundario es cualquier cambio de estado en la aplicación que sea observable fuera de la función llamada. Algo complejo, ¿no? Examinemos un poco este código:

```
public static int funcion(int x){  
    variable_global = 10; //Cambio de valor  
    return x + 10;  
}
```

En este caso la "función" no es una función pura, ya que esta posee efectos secundarios al modificar el valor de una variable externa a la función. Los efectos secundarios se evitan principalmente en la programación funcional para tener como resultado programas mucho más fáciles de comprender y de probar.

2.3 Expresiones lambda (funciones anónimas)

Una **expresión lambda**, también llamada **función lambda**, **función literal** o **función anónima**, es una subrutina definida que no está enlazada a un identificador (es decir, no tiene nombre). Son muy comunes en los lenguajes que implementan programación funcional, y pueden verse como "funciones de un sólo uso". Muy usadas en el manejo de Streams (flujos) y en los eventos de interfaces gráficas. En apartados anteriores ya se han mostrado pero ahora vamos a formalizarlas.

Una **expresión lambda** es un bloque corto de código que toma parámetros y devuelve un valor, parecido a como lo hace un método, pero sin necesidad de que tenga nombre. Consiste en una lista separada por comas de los parámetros formales encerrados entre paréntesis, un símbolo de flecha (->) y un cuerpo. Los tipos de datos de los parámetros siempre se pueden omitir, al igual que los paréntesis si solo hay un parámetro. El cuerpo puede consistir en una sentencia o un bloque de sentencia.

Sintaxis: (parámetros) -> {cuerpo-lambda}

Cuando se tiene un solo parámetro no es necesario utilizar los paréntesis.

Cuando el cuerpo tiene una sola línea no hace falta usar llaves ni poner return en caso de devolver valores.

Veamos algunos ejemplos de expresiones lambda:

```
// sin parámetros
() -> System.out.println("Hola!, que tal...")

// Con un parámetro (este ejemplo es una función de identidad).
a -> a

// Con una expresión y más de 1 parámetro
(a, b) -> a + b

// Con un bloque de código (funciona igual que la anterior)
(a, b) -> { return a + b; }

// Con información de tipo explícita
(long id, String name) -> "id: " + id + ", name:" + name

/* Con múltiples afirmaciones en el cuerpo lambda. Necesita un bloque
de código. Este ejemplo también incluye dos expresiones lambda anidadas
(la primera también es una clausura). */

(id, defaultPrice) -> {
    Optional<Product> product = productList.stream()
                                           .filter(p -> p.getId() == id)
                                           .findFirst();
    return product.map(p -> p.getPrice()).orElse(defaultPrice);
}
```

Limitaciones en Java

Las expresiones lambda en Java 8 poseen las siguientes limitaciones:

- Pueden regresar excepciones comprobadas, pero tales lambdas no funcionarán con las interfaces utilizadas por la API de la colección.
- Las variables que están dentro del alcance donde se declara el lambda solo se pueden acceder dentro del lambda si son efectivamente finales, es decir, si la variable no está mutada dentro o fuera del ámbito lambda.

Ejemplo con interfaces funcionales: Comparator

En Java, se considera interfaz funcional a toda interfaz que contenga un único método abstracto. Por eso, la interfaz “Comparator” del paquete java.util, vista en unidades anteriores, es una interfaz funcional. Solo tiene un método llamado **compare** que toma dos objetos como parámetros y los compara devolviendo un número entero que nos dice qué objeto debe ir primero en una ordenación.

Recordemos su funcionamiento con este ejemplo para comparar 2 String según su longitud de la manera tradicional:

```

public class MyStringComparator implements Comparator<String> {
    @Override
    public int compare(String s1, String s2)
    {
        return Integer.compare(s1.length(), s2.length());
    }
}

// MAIN
List<String> list = Arrays.asList("Hello", "Hi", "Goodbye", "Farewell", "Bye");
MyStringComparator msc = new MyStringComparator();
...
Collections.sort(list, msc);

```

Sin embargo, a partir de Java 8, podemos hacer uso de las expresiones lambda y ahorrar código. Veamos cómo:

```

Comparator<String> lComp = (s1,s2) -> Integer.compare(s1.length(), s2.length());
List<String> list = Arrays.asList("Hello", "Hi", "Goodbye", "Farewell", "Bye");
Collections.sort(list, lComp);

```

Incluso podemos acortar ese código colocando la expresión lambda como segundo parámetro del método Collections.sort:

```

List<String> list = Arrays.asList("Hello", "Hi", "Goodbye", "Farewell", "Bye");
Collections.sort(list, (s1,s2) -> Integer.compare(s1.length(), s2.length()));

```

2.4 Streams

Gracias a la llegada de Java 8, y a la introducción de los streams, las colecciones han aumentado su funcionalidad. Los streams permiten realizar operaciones funcionales sobre los elementos de las colecciones. Prácticamente todas las aplicaciones tienen que trabajar con colecciones (buscar elementos con un determinado valor, ordenarlas, transformar sus datos, etc.) Típicamente, esto se ha hecho con bucles, iterando una y otra vez sobre ellos, repitiendo el mismo código. Además, para hacer el trabajo de forma eficiente, pretendemos utilizar varios núcleos de nuestra CPU, pero es difícil y una fuente de errores. Los data processing streams vienen a solucionar este problema.

Los Streams presentan las siguientes características:

- Operaciones fácilmente paralelizables.
- Estilo declarativo de operaciones.
- Concatenación de operaciones en un pipeline.

Para utilizar los streams sobre una colección, basta con invocar al método `stream()` o `parallelStream()`, en función de si queremos paralelizar las operaciones o no.

A continuación, mostramos un ejemplo de las bondades de los streams donde, a partir de una lista de personas (cada una de ellas tiene un nombre), obtenemos una lista con todos los nombres (en una sola línea de código. Piensa en cómo lo harías sin usar “streams”):

[illegible]

Otro ejemplo sería filtrar una lista de números para obtener solo los pares:

```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5, 6);

List<Integer> pares = numeros.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
```

Como hemos, de una forma muy fácil hemos obtenido todos los nombres así como los pares.

O simplemente:

```
int[] array = {1, 2, 3, 4, 5};  
int sum = Arrays.stream(array).sum();
```

Veamos más ejemplos del uso de Streams

Supongamos que tenemos un ArrayList de objetos de tipo “Persona” llamado “personas”. El siguiente ejemplo obtiene una estructura nueva llamada “adultos”, a partir de “personas” con las que cumplen la condición de tener una edad mayor o igual a 18 años. Además, las ordena por edad.

```
Stream<Persona> adultos = personas.stream()
    .filter(p -> p.getEdad() >= 18)
    .sorted((p1,p2) -> p1.getEdad() - p2.getEdad());
```

Este ejemplo obtiene una estructura nueva llamada “edades”, a partir de “personas” que contiene números enteros, concretamente las edades de cada persona. Sólo las edades.

```
Stream<Integer> edades = personas.stream()
                                .map(p -> p.getEdad());
```

Este ejemplo es similar al anterior, pero sólo almacena las edades que son mayor o igual a 18 años.

```
Stream<Integer> edadesadultos = personas.stream()
    .filter(p -> p.getEdad() >= 18)
    .map(p -> p.getEdad());
```

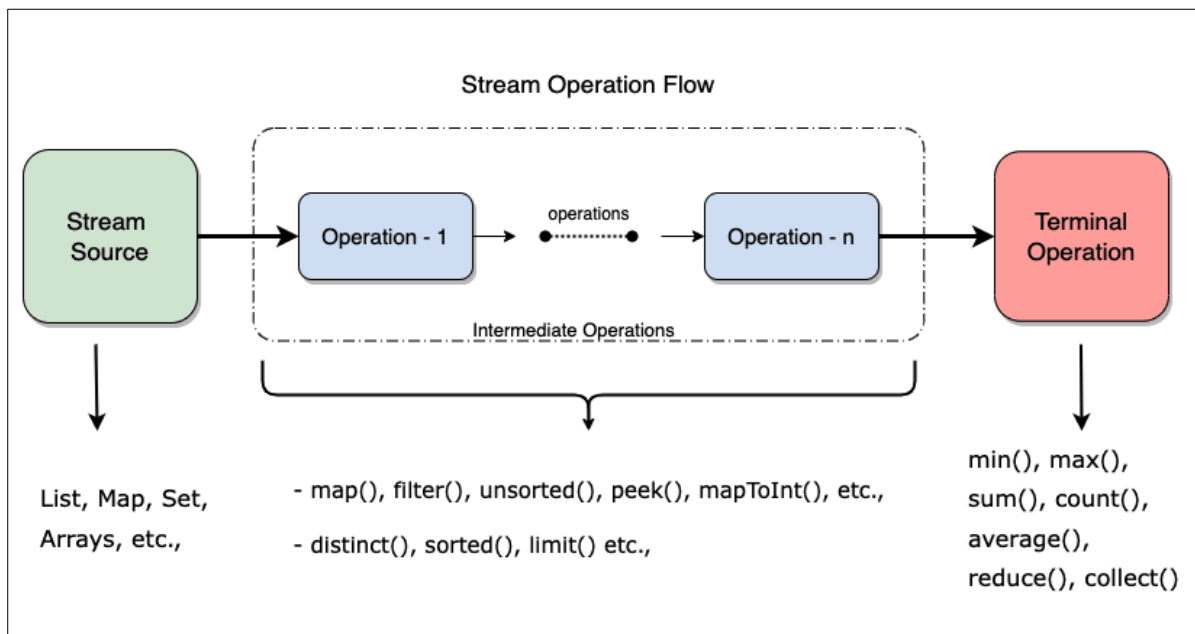
Estudia los siguientes 3 ejemplos y reflexiona, ¿qué consigue cada uno?

```
List<Integer> edadesadultos = personas.stream()
    .filter(p -> p.getEdad() >= 18)
    .map(p -> p.getEdad())
    .collect(Collectors.toList());
```

```
personas.stream()
    .filter(p -> p.getEdad() >= 18)
    .map(p -> p.getEdad())
    .forEach(p -> System.out.println(p));
```

```
double mediaedadadultos =
    .filter(p -> p.getEdad() >= 18)
    .map(p -> p.getEdad())
    .mapToInt(p -> p.getEdad())
    .average()
    .getAsDouble();
```

Stream es una interfaz que permite convertir una fuente de datos (por ejemplo, una colección) en un objeto especial que permite realizar operaciones eficientes sobre esos datos, nunca se modifica la fuente original. A continuación se muestra el flujo de trabajo de las operaciones sobre un "Stream":



Podemos observar 3 fases principalmente, y sus operaciones más destacadas:

- A) Fuente:** los datos se obtienen de una colección, generalmente.
- B) Operaciones intermedias:** cada operación en la canalización se aplica a cada elemento en una secuencia (filtrados, ordenaciones, etc.)
 - **map:** altera los elementos en base a una función y los devuelve alterados. También

permite realizar proyecciones de atributos (multiplicar por 2, obtener un campo de un objeto...)

- **filter**: filtra los elementos a partir de la condición de la función pasada como parámetro (uso de condicionales: equals, etc...)
- **distinct**: devuelve un stream con elementos no repetidos basándose en la comparación entre objetos con la función equals (Object.equals(object)).
- **sorted**: ordena los elementos siguiendo la condición de la función como parámetro (es decir, un Iterator si el objeto no es Comparable).

C) Operación terminal: se realiza al final, puede ser un cálculo, mostrar en pantalla o almacenar los resultados en una estructura de datos **nueva**.

- **forEach**: realiza la acción para cada elemento del stream.
- **collect**: ejecuta lo que se llama “mutable reduction”, que consiste en acumular los resultados en una Collection a medida que los va procesando en la pipeline del stream.
- **reduce**: permite hacer una reducción de los resultados del stream, lo cual consiste en acumular el resultado en un resultado resumido de la entrada, por ejemplo, encontrar la suma de un stream de enteros. También hay otras operaciones que utilizan la operación reduce en background y son terminales, como sum().

La mayor similitud o manera de imaginarse los Streams con otro término de la informática es con el lenguaje estructurado más utilizado en base de datos relacionales, el Structured Query Language (SQL). Imagina que, dada una base de datos, nos piden:

«Seleccionar los dos primeros modelos distintos de los vehículos con los kilómetros igual o menor a 100000»

Con un poco de conocimiento sobre SQL, enseguida hallaríamos la sentencia adecuada:

```
SELECT DISTINCT modelo
FROM Vehiculos
WHERE kilometros <= 100000 LIMIT 2;
```

Esto es un lenguaje declarativo; especificamos qué queremos pero no tenemos que preocuparnos de implementar manualmente cómo conseguirlo. Eso es posible también mediante Java con la programación funcional (que es declarativa). Veamos el equivalente a la expresión SQL anterior mediante Java (sobre un List no una BD):

```
List<String> modelosDistintos = vehiculos.stream()
    .filter(v -> v.getKilometros() <= 100000)
    .map(Vehiculo::getModelo)
    .distinct()
    .limit(2)
    .collect(Collectors.toList())
```

Piensa en cómo harías lo anterior de forma “imperativa” (mucho más código, obviamente).

3. OTROS LENGUAJES Y PARADIGMAS

Los lenguajes multiparadigma son aquellos que soportan la combinación de 2 o más paradigmas, permitiendo crear programas usando más de un estilo de programación. El objetivo en el diseño de estos lenguajes es permitir a los programadores utilizar el mejor paradigma para cada trabajo, admitiendo que ninguno resuelve todos los problemas de la forma más fácil y eficiente posible. Hoy en día la mayoría de lenguajes más usados son multiparadigma.

Ejemplos:

- **C++:** combina la programación estructurada con la orientación a objetos.
- **Python/PHP:** son orientados a objetos, reflexivos, estructurados y funcionales.
- **Java:** es fundamentalmente orientado a objetos ("casi" todo en Java es un objeto), incluye el imperativo y también el funcional desde la versión 8.
- **Javascript:** orientado a objetos (basado en prototipos), imperativo y con soporte para funcional.

3.1 Características de otros lenguajes multiparadigma con enfoque web

- **Python:** multipropósito.

En el caso web se usa fundamentalmente en backend (con Django). También existe una implementación para ser usado en frontend: Brython. Brython se ha diseñado para reemplazar a JavaScript como lenguaje de scripting para web. Por lo tanto, es una implementación de Python adaptada a un entorno HTML5, es decir, con una interfaz con los objetos y eventos DOM. Sin embargo, al ser un traductor de Python a Javascript, está algo limitado.

- **JavaScript:** sólo web (fundamentalmente frontend) e interpretado en la máquina cliente. NodeJS es la versión para backend. Tiene multitud de librerías y frameworks extras como: jQuery, React JS, AngularJS, Vue.js, etcétera...

Orientado a objetos, ya que maneja objetos dentro de la página Web sobre los cuales podemos definir diferentes eventos. Dichos objetos facilitan la programación de paginas interactivas a través del uso del DOM (Document Object Model). El DOM es esencialmente una interfaz que proporciona un conjunto estándar de objetos para representar documentos HTML, XHTML y XML, un modelo estándar sobre cómo pueden combinarse dichos objetos, y una interfaz estándar para acceder a ellos y manipularlos. A través del DOM los programas pueden acceder y modificar el contenido, estructura y estilo de los documentos HTML y XML, que es para lo que se diseñó principalmente. Responde a eventos en tiempo real. Eventos como presionar un botón, pasar el puntero del ratón sobre un determinado texto o el simple hecho de cargar la página o caducar un tiempo. Con esto podemos cambiar totalmente el aspecto de la página al gusto del usuario, evitándonos tener en el servidor un página para cada gusto, hacer cálculos en base a variables cuyo valor es determinado por el usuario, etc.

- **PHP:** sólo web (backend) y ejecutado en la máquina servidor por un intérprete, la cual genera como resultado una web que se envía al cliente.

Es un lenguaje muy extendido y de larga tradición en el ámbito web. Es un lenguaje de alto nivel, orientado a objetos y básicamente imperativo, aunque incluye un pequeño soporte al paradigma declarativo estilo funcional. Permite el uso de patrones de diseño, como el MVC (Modelo Vista Controlador).

3.2 El tipado de datos

El tipado es una característica importante y, al hablar de lenguajes multiparadigma, debemos detallarlo. El tipado, o sistema de tipos, define la forma en que un lenguaje de programación clasifica los valores y las expresiones, definiendo cómo se pueden manipular estos tipos y cómo interactúan. Un tipo de dato indica un conjunto de valores que tienen el mismo significado genérico o propósito (aunque algunos tipos, como los tipos de datos abstractos y tipos de datos función tal vez no representen valores en el programa que se está ejecutando). Los sistemas de tipificación varían significativamente entre lenguajes, siendo quizás las más importantes variaciones las que estén en sus propias implementaciones de la sintáctica en tiempo de compilación y la operativa en tiempo de ejecución.

Hay 2 clasificaciones fundamentales: **fuerte ó débil, estático ó dinámico**; y sus combinaciones. Veamos cada una:

Tipado fuerte

- Java
- C#
- Python

Un lenguaje es fuertemente tipado si no se permiten violaciones de los tipos de datos, es decir, dado el valor de una variable de un tipo concreto, no se puede usar como si fuera de otro tipo distinto a menos que se haga una conversión. Cuando un lenguaje es fuertemente tipado, el programador debe explícitamente convertir un tipo de datos en otro. Esto se conoce como conversión o moldeado de tipos (typecasting) y también debe estar soportado por el compilador.

Tipado débil

- JavaScript
- PHP

Los lenguajes de programación no tipados o débilmente tipados no controlan los tipos de las variables que declaran, de este modo, es posible usar variables de cualquier tipo en un mismo escenario. Por ejemplo, una función puede recibir como parámetro un valor entero, cadena de caracteres, flotante, etc. No hay que confundir el término con los lenguajes de tipos dinámicos, en los que los tipos de las variables se deciden en tiempo de ejecución, si bien es cierto que muchos lenguajes de programación de tipos dinámicos (en

los que no se declaran los tipos de datos) son también no tipados.

No son necesarias las conversiones de tipos, por lo tanto el siguiente ejemplo Javascript es correcto:

```
var a=3;  
var b="4";  
var c=a+b; // c contiene la cadena "34"
```

Tipado estático

Se dice que lenguaje de programación usa un tipado estático cuando la comprobación de tipos se realiza durante la compilación, y no durante la ejecución. Ejemplos de lenguajes que usan tipado estático son: C, C++, Java y Haskell. Comparado con el tipado dinámico, el estático permite que los errores de tipificación sean detectados antes, y que la ejecución del programa sea más eficiente y segura.

```
int var; //declara una variable que sólo acepta enteros
```

Tipado dinámico

Los tipos de las variables se deciden en tiempo de ejecución. Por tanto, una variable puede tomar valores de distintos tipos. La mayoría de lenguajes de tipado dinámico son lenguajes interpretados, como Python o Ruby. Un lenguaje que no es dinámicamente tipado se dice que es de tipado estático, o estáticamente tipado. Esta separación se suele confundir con la diferencia entre lenguajes fuertemente tipados y lenguajes débilmente tipados.

```
var = 3; #Aquí la variable "var" adopta el tipo de datos entero  
var = 2.3; # Aquí se adapta al tipo de datos decimal o coma flotante
```

Veamos cómo se clasifican algunos de los lenguajes más conocidos:

- **Java, C#:** Tipado fuerte y estático.
 - **Python:** Tipado fuerte y dinámico.
 - **Javascript / PHP:** Tipado débil y dinámico.
-

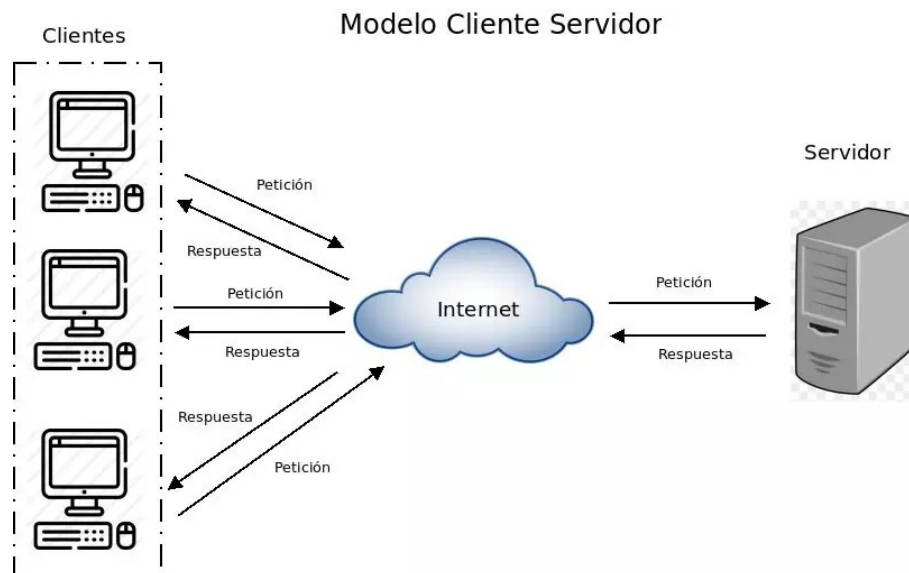
4. INTRODUCCIÓN AL DESARROLLO WEB BACKEND CON JAVA

El ecosistema de tecnologías para desarrollo web es inabarcable, por ello, en este apartado vamos a realizar un acercamiento general, y también trataremos de ver cómo Java, que es el lenguaje que hemos empleado en este módulo, se puede emplear para desarrollo de aplicaciones web.

El modelo de desarrollo web se apoya en lo que se conoce como arquitectura cliente-servidor, que define un patrón de arquitectura donde existen 2 actores (cliente y servidor) de forma que el primero es quién se conecta con el segundo para solicitar algún servicio. En el caso que nos ocupa, el desarrollo web, los clientes solicitan que se les sirva una web para visualizarla e interactuar con ella (procedente de una aplicación web), aunque también es posible solicitar información si hablamos del caso de los servicios web, aspecto que se verá en otros módulos. Sin embargo, en ambos casos aparece el mismo escenario donde un servidor se encuentra ejecutándose ininterrumpidamente a la espera de que múltiples clientes realicen una solicitud.

Normalmente a la solicitud que hacen los clientes al servidor se le llama petición (**request**) y a lo que el servidor devuelve a dicho cliente le llamamos respuesta (**response**).

También hay que tener en cuenta que esta arquitectura cliente-servidor plantea la posibilidad de numerosos clientes atendidos por un mismo servidor. Es decir, el servidor será un software **multitarea** que será capaz de atender peticiones simultáneas de numerosos clientes.



Front-end, Back-end, Full stack

También teniendo en cuenta en que lado se ubican las tecnologías y para qué se utilizan éstas, actualmente se habla de 3 perfiles diferenciados en el ámbito del desarrollo web:

- **Front-end:** parte del desarrollo que se encarga del diseño y maquetación de la aplicación web utilizando tecnologías como **HTML, CSS y Javascript** (y sus **frameworks**). En este caso debe preocuparse también de la correcta presentación en cualquier tipo de dispositivo (*responsive*) e incluso del posicionamiento en buscadores (SEO).
- **Back-end:** parte del desarrollo que se encarga del software en el lado servidor utilizando tecnologías como **PHP, Java (JSP) y Python**. También se encarga de la administración del servidor de aplicaciones y del uso y acceso a la base de datos.
- **Full stack:** es un perfil que engloba los dos anteriores. En este caso el desarrollador quizás no es un experto de ninguna tecnología concreta pero tiene amplios conocimientos de todo el conjunto y es capaz de colaborar en cualquiera de las partes. Este perfil requiere años de experiencia tanto en front-end como en back-end y una sólida formación técnica en constante reciclaje.

4.1 Tecnologías backend en Java

Java ha evolucionado significativamente y hoy en día existen múltiples tecnologías y frameworks que facilitan el desarrollo backend robusto y escalable. Algunos de los más destacados son (aunque hay más):

- **Spring Framework:** es el framework más popular para el desarrollo web con Java. Spring ofrece un ecosistema completo para construir aplicaciones web, desde el manejo de dependencias con Spring Core, la creación de servicios REST con Spring MVC, hasta la integración con bases de datos mediante Spring Data. También incluye módulos como Spring Boot, que simplifica la configuración y el despliegue de aplicaciones.
- **Jakarta EE** (antiguamente Java EE): es un conjunto de especificaciones que extiende Java SE con funcionalidades adicionales para aplicaciones empresariales. Incluye tecnologías como Servlets, JSP, JPA (para acceso a datos), CDI (para inyección de dependencias), y muchos otros componentes esenciales para el desarrollo de aplicaciones empresariales robustas y escalables. Es un stack obsoleto pero interesante para aprender cómo se usa Java con aplicaciones web.

Para ilustrar con un ejemplo el funcionamiento del backend con Java, lo haremos con Jakarta EE; aunque a día de hoy se usa Spring, éste es un framework más complicado de usar y excede los objetivos de realizar una introducción al backend, de ahí que usaremos Jakarta EE.

Jakarta EE (JSP / Servlets)

En Jakarta EE tenemos estas 2 tecnologías que trabajan con Java para la parte web:

- **JavaServer Pages (JSP)**
 - **Java Servlets**
-

JavaServer Pages (también conocido como **JSP**) es una tecnología para la creación de páginas web dinámicas del lado servidor. Al igual que PHP, su código se escribe incrustado junto con el HTML de la página web y es el servidor de aplicaciones, en este caso, quién debe procesarlo para generar la página web resultante, en HTML, que se enviará al lado cliente.

Por otra parte, **Java Servlets** es una tecnología que también se puede utilizar para crear contenido web dinámico pero que además extiende su funcionalidad a la posibilidad de conectar esas webs dinámicas con otro contenido accesible a través de Internet. En ocasiones se utiliza junto con JSP para crear aplicaciones web más complejas.

Los **servlets** son más adecuados para aplicaciones web grandes y complejas que necesitan un mayor control y flexibilidad en el manejo de las solicitudes y respuestas HTTP, mientras que **JSP** es más adecuado para aplicaciones web más pequeñas y sencillas en las que se requiere una presentación dinámica de la información.

En última instancia, la elección entre utilizar servlets o JSP depende del tipo de aplicación web que se esté desarrollando y de los requisitos específicos de esa aplicación. A menudo se utilizan ambos en combinación para aprovechar al máximo las ventajas de cada uno.

4.2 Servlets

Un **Servlet** es una clase Java usada para dar servicio web y para extender la capacidad de aplicaciones basadas en el modelo cliente-servidor, y que utilizan el protocolo HTTP basado en la interacción de ambos extremos por medio de una petición y una respuesta. Los servlets son una potente herramienta para la generación de contenido dinámico en la Web.

El Contenedor de Servlets (a veces llamado servidor) es el componente encargado de la gestión de los Servlets (instanciar, acceso, destrucción...), y el que controla su ciclo de vida. Dicho contenedor de Servlets será un programa adicional que hay que instalar para dar soporte a este tipo de aplicaciones Java en entornos J2EE. Algunos de estos "contenedores" disponibles son MacroMedia JRun, Bea WebLogic, Sun One, GlassFish y uno de los más populares: Apache Tomcat.

Por lo tanto, es requisito indispensable tener instalado un JDK actualizado previamente a la instalación de Tomcat, ambos necesarios para escribir y probar tanto servlets como páginas JSP, elementos habituales de las aplicaciones J2EE.

Ejemplo con servlets

Realizaremos una página HTML (Param.html) que incluirá un formulario a rellenar por el usuario (cliente). Cuando éste pulse el botón de "Enviar", se llamará a un servlet (ParamServlet.java) que recogerá los datos tecleados y generará una "página dinámica" como respuesta con los datos, y que será mostrada de nuevo al cliente que desencadenó el proceso. Para que exista un servlet atento a las peticiones que le vayan llegando, tendremos instalado e iniciado el contenedor de servlets (Tomcat) al que configuraremos un archivo (web.xml) donde deben figurar todos los servlets disponibles para su ejecución.

Antes de escribir ninguno de los tres ficheros, es importante señalar unos cuantos requisitos que hay que cumplir para escribir servlets. Una vez instalado el contenedor, existirá un directorio llamado "webapps". Es en él donde colocaremos todas las aplicaciones web que hagamos (servlets, jsp, ejb, etc). Lo ideal es que nos creemos otro directorio dentro del mismo y allí dentro, otra estructura de directorios que incluirán uno llamado "WEB-INF" y dentro de ese, otro con el nombre "classes" (exactamente, incluyendo las mayúsculas, guiones, etc. Para la práctica se creó el directorio "misServlets", quedando la estructura siguiente:

```
+---Tomcat 5.5
|
+---webapps
|
+---misServlets
|   Param.html
|
+---WEB-INF
|   web.xml
|
+---classes
    ParamServlet.class
```

Veamos la página html (Param.html):

```
<!DOCTYPE html>
<html>
<head>
<title>Introduce tus datos</title>
<style>
  body {
    background-color: #ccccff;
  }
  form {
    margin: 0 auto;
    width: 50%;
    padding: 20px;
    border: 1px solid black;
    border-radius: 10px;
  }
  label {
    display: block;
    font-weight: bold;
    margin-bottom: 5px;
  }
  input[type=text] {
    width: 100%;
    padding: 8px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box;
  }
  input[type=submit] {
    background-color: #4CAF50;
    color: white;
    padding: 8px 16px;
```

```

        border: none;
        border-radius: 4px;
        cursor: pointer;
        margin-top: 10px;
    }
    input[type=submit]:hover {
        background-color: #3e8e41;
    }
</style>
</head>
<body>
    <form action="http://localhost:8080/misServlets/ParamServlet" method="post">
        <h2>Introduce tus datos:</h2>
        <hr>
        <label for="nombre">Nombre:</label>
        <input type="text" id="nombre" name="NOM">
        <label for="edad">Edad:</label>
        <input type="text" id="edad" name="EDA">
        <input type="submit" value="Enviar">
        <hr>
        <p>Este formulario HTML invocará a un servlet</p>
    </form>
</body>
</html>

```

Si lo visualizamos veríamos algo como esto:

Observemos unos cuantos puntos importantes. En la página incluimos un formulario rellenable. Comienza con **<FORM...>** y termina con **</FORM>**. En él se indica que los datos se enviarán mediante el método **POST** al servlet, cuyo nombre es **"ParamServlet"** y está en un directorio llamado **"misServlets"**, dentro del contenedor Tomcat (recuerda que el contenedor es el "servidor"). Dicho contenedor está en el mismo ordenador ya que el nombre del servidor que lo alberga es **"localhost"**. Esa llamada al servlet se producirá en cuando el usuario pulse el botón "Enviar" (**<input type=submit>**). La página la colocamos en la carpeta "misServlets" (o el nombre decidido para este fin).

Veamos el servlet (ParamServlet.class):

Pongamos ahora el código del servlet, no sin antes indicar que hay que incluirle la librería **servlet-api.jar**, que se encuentra en la carpeta de *Tomcat* `..\common\lib`. Se puede establecer el classpath, copiarla, o adjuntarla al proyecto. En Eclipse es con las opciones Project → Properties → Java Build Path → Libraries y botón "**Add external jar**" para, navegando, incluir el fichero mencionado.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ParamServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    public void doPost(HttpServletRequest req, HttpServletResponse res) throws
ServletException, IOException {
        // Obtenemos un objeto PrintWriter para enviar respuesta
        res.setContentType("text/html");
        PrintWriter pw = res.getWriter();
        pw.println("<HTML><HEAD><TITLE>Leyendo parámetros</TITLE></HEAD>");
        pw.println("<BODY BGCOLOR=\"#CCBBAA\">");
        pw.println("<H2>Leyendo parámetros desde un formulario html</H2><P>");
        pw.println("<UL>\n");
        pw.println("Te llamas " + req.getParameter("NOM") + "<BR>");
        pw.println("y tienes " + req.getParameter("EDA") + " años<BR>");
        pw.println("</BODY></HTML>");
        pw.close();
    }
}
```

Una vez compilado este programa, se coloca en la carpeta `classes` de la estructura que hicimos.

Finalmente, el descriptor

Por último nos queda escribir el "descriptor de despliegue", elemento necesario antes de que pueda invocarse cualquier servlet y que describe cada uno de ellos. Este fichero se llama siempre **web.xml**, por lo que podemos copiarlo de carpetas similares. Luego lo editamos a nuestro gusto y lo dejamos más o menos así:

```
<?xml version="1.0" encoding="ISO-8859-1"?>

<web-app xmlns="http://java.sun.com/xml/ns/j2ee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd"
    version="2.4">

    <display-name>Servlets ejemplo</display-name>
    <description>
        Servlets de ejemplo para practicar
    </description>
```

```
<!-- Publicamos nuestro servlet aqui -->

<servlet>
  <servlet-name>ParamServlet</servlet-name>
  <servlet-class>ParamServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>ParamServlet</servlet-name>
  <url-pattern>/ParamServlet</url-pattern>
</servlet-mapping>

<!-- Fin de la publicacion de servlets -->

</web-app>
```

Es importante ser cuidadoso con la sintaxis, ya que los errores por fallos en este archivo a veces son difíciles de detectar. Obsérvese cómo incluimos dos etiquetas **<servlet>** y **<servlet-mapping>** que dan a conocer este servlet al contenedor. Una vez que lo iniciemos estará disponible para ser invocado.

Ejecución y resultados

Si todo lo anterior ha funcionado, ya podemos abrir nuestro navegador favorito y teclear en la barra de navegación

`http://localhost:8080/misServlets/Param.html`

Si pusimos la página donde se indicó (misServlets) no debería haber problemas. Las dificultades pueden venir al pulsar el botón "Enviar" del formulario, momento en el cual se envía la "petición" del usuario a Tomcat. El contenedor recibe la petición y buscará el servlet en la lista de los posibles en nuestro descriptor web.xml. Si todo está bien, será encontrado e invocado, ejecutando el método **POST** (que es como el main), ya que así se indicaba en la etiqueta **<FORM>** de la página HTML.

El método **doPost()** tiene dos argumentos:

1. El primero representa la "petición" del cliente (usuario remoto) con el nombre de **HttpServletRequest**.
2. El segundo es la "respuesta" que generará el servlet hacia el cliente en la forma del objeto **HttpServletResponse**.

El servlet, lo primero, ha de indicar el tipo de información que se va a emitir (html) y luego forma un objeto asociado con el navegador web del cliente para imprimir allí los resultados, por medio del método **getWriter()**. Extraerá los datos de los cuadro de texto (el nombre y la edad) mediante los métodos **getParameter()** y se dedicará a formar la página HTML dinámica. Una vez terminada, se cierra (close) y es enviada automáticamente.

4.3 JSP

JSP (JavaServer Pages) es una tecnología que permite incluir código Java en páginas web HTML. El denominado contenedor JSP (que sería un componente del servidor web) es el encargado de

coger la página, sustituir el código Java que contiene por el resultado de su ejecución, y enviarla al cliente. Así, se pueden diseñar fácilmente páginas con partes fijas y partes variables. El siguiente es un ejemplo muy sencillo de página JSP:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
<html>
  <head>
    <title>Mi primera página JSP</title>
  </head>
  <body>
    <h1> Hoy es: <%= new java.util.Date() %> </h1>
  </body>
</html>
```

Para ejecutar la página anterior basta con colocarla en una aplicación web (por ejemplo, en Tomcat, dentro de webapps/ROOT). No es necesario que sea en un directorio específico como ocurre con los servlets, sino que puede ir en cualquier directorio en el que se colocaría normalmente un HTML.

Aunque JSP y servlets parecen a primera vista tecnologías distintas, en realidad el servidor web traduce internamente el JSP a un servlet, lo compila y finalmente lo ejecuta cada vez que el cliente solicita la página JSP. Por ello, en principio, JSPs y servlets ofrecen la misma funcionalidad, aunque sus características los hacen apropiados para distinto tipo de tareas.

Los **JSP** son **mejores** para generar páginas con gran parte de contenido estático. Un servlet que realice la misma función debe incluir gran cantidad de sentencias del tipo `out.println()` para producir el HTML.

Por el contrario, los **servlets** son **mejores** en tareas que generen poca salida, datos binarios o páginas con gran parte de contenido variable. En proyectos más complejos lo recomendable es combinar ambas tecnologías: los servlets para el procesamiento de información y los JSP para presentar los datos al cliente.

Ejemplo anterior pero con servlet:

```
public void _jspService(HttpServletRequest request, HttpServletResponse response)
throws java.io.IOException, ServletException {
    JspWriter out = null;
    response.setContentType("text/html;ISO-8859-1");
    out.println("<!DOCTYPE HTML PUBLIC \"/>
```